

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

CO1-AT1-Analytical Problem Solving

Register Number	192312428
Name	B.Bhavana

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. (BL3)

Assessment Tool Weightage: 40%

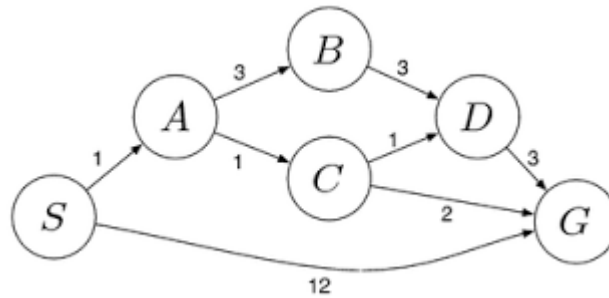
QUESTIONS: 5

Total Marks: 50

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.
2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
 - i. What are the percept's for this agent?
 - ii. Characterize the operating environment.
 - iii. What are the actions the agent can take?
 - iv. How can one evaluate the performance of the agent?
 - v. What sort of agent architecture do you think is most suitable for this agent? Why?
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.
4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.
5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path

from the starting warehouse S to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Code of Conduct:

I B. Bhavana (192312428) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B. Bhavana

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem	10
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method	10
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process	12.5
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations	10
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation	7.5

Question 1: The Water Jug Problem

Problem: Given a 4-gallon jug and a 3-gallon jug, neither marked, and a pump, obtain exactly 2 gallons of water in the 4-gallon jug.

Problem Formulation

State representation: (x, y) , where x = amount of water in the 4-gallon jug (0-4) and y = amount of water in the 3-gallon jug (0-3).

Initial state: $(0, 0)$ - both jugs empty.

Goal state: $(2, y)$ for any value of y - the 4-gallon jug contains exactly 2 gallons.

Production Rules (Operators)

S.no	Rule (Current State $\rightarrow$ Next State)	Condition
1	$(x, y) \rightarrow (4, y)$	Fill the 4-gallon jug from the pump, if $x < 4$
2	$(x, y) \rightarrow (x, 3)$	Fill the 3-gallon jug from the pump, if $y < 3$
3	$(x, y) \rightarrow (0, y)$	Empty the 4-gallon jug on the ground, if $x > 0$
4	$(x, y) \rightarrow (x, 0)$	Empty the 3-gallon jug on the ground, if $y > 0$
5	$(x, y) \rightarrow (0, x+y)$	Pour 4-gallon jug into 3-gallon jug, if $x+y \leq 3$
6	$(x, y) \rightarrow (x+y, 0)$	Pour 3-gallon jug into 4-gallon jug, if $x+y \leq 4$
7	$(x, y) \rightarrow (4, y-(4-x))$	Pour 3-gallon jug into 4-gallon jug until 4-jug is full, if $x+y > 4$
8	$(x, y) \rightarrow (x-(3-y), 3)$	Pour 4-gallon jug into 3-gallon jug until 3-jug is full, if $x+y > 3$

Solution Path

Step	4-gallon jug	3-gallon jug	Action performed
0	0	0	Initial state
1	4	0	Fill the 4-gallon jug from the pump
2	1	3	Pour water from the 4-gallon jug into the 3-gallon jug
3	1	0	Empty the 3-gallon jug on the ground
4	0	1	Pour water from the 4-gallon jug into the 3-gallon jug

5	4	1	Fill the 4-gallon jug from the pump
6	2	3	Pour water from 4-gallon jug into 3-gallon jug until it is full

Result: After step 6, the 4-gallon jug contains exactly 2 gallons of water - the goal state (2, 3) has been reached.

Question 2: Mars Rover Agent

The Mars Rover is an agent that perceives its Martian environment through sensors and acts upon it through actuators in order to analyse and collect rock/soil samples.

i. Percepts of the Agent

- Camera / stereo-vision images of the terrain and rock formations
- Spectrometer and chemical-sensor readings of soil and rock composition
- GPS / inertial navigation data giving current position and orientation
- LIDAR / ultrasonic sensor readings for obstacle distance and terrain slope
- Temperature, radiation, and atmospheric pressure sensor readings
- Battery / solar-panel power level readings
- Commands and mission updates received from ground control on Earth

ii. Characterisation of the Operating Environment

Property	Characterisation	Justification
Observability	Partially observable	Sensors cannot perceive the entire Martian terrain or subsurface at once
Number of agents	Single agent	Only the rover acts in the environment (no competing agents)
Determinism	Stochastic	Wheel slippage, dust storms and uncertain terrain make outcomes unpredictable
Episodic / Sequential	Sequential	Current movements and decisions affect future actions and mission outcome
Static / Dynamic	Dynamic	Weather, lighting and terrain conditions change over time, even while the rover 'thinks'
Discrete / Continuous	Continuous	Position, motor speed, and sensor readings vary continuously

iii. Actions the Agent Can Take

- Move forward / backward, turn left / right, climb over or steer around obstacles
- Extend robotic arm and drill into rock or soil
- Collect and store a sample in the sample chamber
- Capture images and record spectrometer/chemical readings
- Transmit collected data to an orbiter or directly to Earth
- Orient solar panels and manage power consumption / recharge
- Halt or replan a path when a hazard is detected

iv. Evaluating the Agent's Performance

- Number of valid rock/soil samples successfully collected and analysed
- Distance travelled safely without collision, tipping over, or getting stuck
- Accuracy and completeness of scientific data transmitted to Earth

- Efficient use of battery power / solar energy over the mission
- Time taken to reach designated target sites within the mission schedule
- Successful avoidance of hazards (steep slopes, sand traps, sharp rocks)

v. Most Suitable Agent Architecture

A Learning, Goal-based (Utility-based) agent built on a model-based reflex core is the most suitable architecture for the Mars Rover, for the following reasons:

- Model-based: the rover must maintain an internal model of the terrain since the environment is only partially observable.
- Goal/Utility-based: the rover must plan paths and choose actions that maximise scientific value (best samples) while minimising risk and energy cost - a pure reflex agent cannot make such trade-offs.
- Learning: due to the huge communication delay with Earth, the rover must adapt its behaviour and improve hazard-detection/navigation from past experience without waiting for human intervention.

Question 3: The 8-Queens Problem - Problem Formulation

Goal: Place 8 queens on an 8×8 chessboard so that no two queens attack each other (no two queens share the same row, column, or diagonal).

There are two main kinds of formulation.

- An incremental formulation involves operators that *augment the state description*,
– starting with an empty state;
– for the 8-queens problem, this means that each action adds a queen to the state.
- A complete-state formulation starts with all 8 queens on the board and moves them around.

The first incremental formulation one might try is the following:

- **States:** Any arrangement of 0 to 8 queens on the board is a state.
- **Initial state:** No queens on the board.
- **Actions:** Add a queen to any empty square.
- **Transition model:** Returns the board with a queen added to the specified square.
- **Goal test:** 8 queens are on the board, none attacked

A **better formulation** would prohibit placing a queen in a square that is already attacked:

- **States:** All possible arrangements of n queens ($0 < n < 8$), one per column in the leftmost n columns, with no queen attacking another.
- **Actions:** Add a queen to any square in the leftmost empty column such that it is not attacked by any other queen.

Search Strategy

A depth-first backtracking search over the incremental (column-by-column) formulation is the standard strategy: place a queen in the current column, and if no safe square exists, backtrack to the previous column and try the next available row. This continues until all 8 queens are placed validly (goal reached) or the search space is exhausted (no solution from that branch).

Question 4: Cab-Booking Agent

Type of Problem-Solving Agent

The cab-booking application is best modelled as a Goal-based (Utility-based) problem-solving agent.

- Goal-based, because the agent's objective is clearly defined: transport the customer from the current location to the destination.
- Utility-based, because among several possible solutions (mini, micro, sedan, shared, prime, etc.) the agent must choose the option that best balances the customer's comfort, cost, and travel time - i.e. it maximises a utility function rather than just reaching any goal state.

The agent formulates the trip as a search problem (current location = initial state, destination = goal state, road network = state space) and then ranks the available cab types using a utility function before presenting/booking the best option.

Pseudocode

function BOOK_CAB_AGENT(current_location, destination, preferences):

// 1. Formulate the problem

```
problem <- FORMULATE-PROBLEM(
    initial_state = current_location,
    goal_state   = destination,
    state_space  = road_network)
```

// 2. Search for the best route

```
route <- SEARCH(problem)      // e.g. UCS / A* on the road network
distance, duration <- ROUTE-METRICS(route)
```

// 3. Evaluate every available cab type

```
cab_types <- {mini, micro, sedan, shared, prime, ...}
```

for each cab in cab_types:

```
    fare[cab] <- ESTIMATE-FARE(cab, distance, duration)
    eta[cab]   <- ESTIMATE-ARRIVAL-TIME(cab, current_location)
    comfort[cab] <- COMFORT-LEVEL(cab)
    utility[cab] <- UTILITY-FUNCTION(fare[cab], eta[cab],
                                     comfort[cab], preferences)
```

// 4. Select and confirm the best option

```
best_cab <- ARGMAX(utility)
CONFIRM-BOOKING(best_cab, route)
return best_cab, route, fare[best_cab], eta[best_cab]
```

Question 5: Uniform Cost Search (UCS) for the Logistics Network

Problem: Find the least-cost delivery path from the starting warehouse S to the destination warehouse G on the given weighted graph, using Uniform Cost Search.

Weighted Graph (edges and travel costs)

Edge	Cost
S → A	2
S → B	5
A → C	4
A → D	7
B → D	3
B → G	9
C → G	6
D → G	1

UCS Algorithm (outline)

- Insert the start node S into a priority queue (frontier) with path cost $g(S) = 0$.
- Repeatedly remove the node with the lowest path cost $g(n)$ from the frontier.
- If it is the goal node G, stop - the least-cost path has been found.
- Otherwise, expand it: for each neighbour, compute the new path cost. If the neighbour is new, or a cheaper path to it has been found, update it in the frontier (and record its parent).

- Add the expanded node to the explored set so it is not re-expanded with a higher cost.

Step-by-Step Trace

Step	Node expanded	Frontier after expansion (node: cumulative cost)
1	S (0)	A:2, B:5
2	A (2)	B:5, C:6 (2+4), D:9 (2+7)
3	B (5)	C:6, D:8 (5+3, replaces 9), G:14 (5+9)
4	C (6)	D:8, G:12 (6+6, replaces 14)
5	D (8)	G:9 (8+1, replaces 12)
6	G (9)	Goal reached - stop

Result

Least-cost path: $S \rightarrow B \rightarrow D \rightarrow G$

Total cost = 5 ($S \rightarrow B$) + 3 ($B \rightarrow D$) + 1 ($D \rightarrow G$) = 9

This is cheaper than the path $S \rightarrow A \rightarrow C \rightarrow G$ (cost = $2+4+6 = 12$) and $S \rightarrow A \rightarrow D \rightarrow G$ (cost = $2+7+1 = 10$), confirming that UCS correctly returns the globally least-cost route from S to G, even though it required expanding nodes out of the order in which they were first discovered.